1 Overview and objectives

For this task, a genetic program (GP) was developed and evaluated for the purpose of forecasting energy demands. The output of the genetic program is a function that forecasts the expected energy load 15 minutes into the future. Individuals of varying fitness levels were produced to solve this problem.

2 Dataset and prepossessing

The initial dataset included the following fields:

- utc_timestamp:
- Electricity_load
- Residential_electricity_price
- Residential_solar_generation
- Residential_wind_generation
- Temperature
- Relative Humidity

For the task, a time-series forecasting model is required. it was decided that for forecasting purposes, only the current load, time and date, as well as a number of lag loads (previous loads leading up to that load), would be used, as well as a previous day load.

The following is an example of a given input and output row of the data.

Input row:

```
utc_timestamp,Electricity_load,Residential_electricity_price
    ,Residential_solar_generation,Residential_wind_generation
    ,Temperature,Relative Humidity
31/12/2014 23:00,0.045,0,0,0,9.8,85.8
```

Output row:

```
load,load_n1,load_n2,load_n3,load_n4,load_n5,load_n6,
    load_prev_day
0.7808219178082192, 0.541095890410959, 0.6917808219178083,
    0.6917808219178083, 0.5547945205479452,
    0.6712328767123288, 0.6643835616438357,
    0.5547945205479452
```

It is hypothesised that only the current and previous loads will turn out to be the strongest predictor of the future load, and, perhaps, the load at the same time as the previous day (which may act as some anchoring value, since load fluctuates on a daily cycle).

The Electricity_load field was processed in the following way:

Processing	Example output
Min-max scaling performed, such that each value is between 0 and 1. Additionally, 6 lag loads and the previous day load were produced from this column. $t-96$ was used for the previous day, since there are 96 15-minute intervals in a day. The following formulas were used: • $load = \frac{curr\_load - min\_load}{max\_load - min\_load}$ • $load\_n1 = previousLoads.at(t-1)$ • $load\_n2 = previousLoads.at(t-2)$ • $load\_n3 = previousLoads.at(t-3)$ • $load\_n4 = previousLoads.at(t-4)$ • $load\_n5 = previousLoads.at(t-5)$ • $load\_n6 = previousLoads.at(t-6)$ • $load\_prev\_day = previousLoads.at(t-96)$	load: 0.780, load_n1: 0.541, load_n2: 0.6917, load_n3: 0.691, load_n4: 0.554, load_n5: 0.671, load_n6: 0.664, load_prev_day: 0.554

3 Terminal and function set representation

Function set: F = { +, -, *, /, sqr } Terminal set: T = { R, load, load_n1, load_n2, load_n3, load_n4, load_n5, load_n6, load_prev_day }. Range of R [-2, 2]

The function set included the most common operators to produce straight line and hyperbolic functions. The current load and 6 lag loads were included in the terminal set, as well as the previous day load and any floating point number between 0 and 2 (this optimal range was discovered through experimentation and parameter tuning). Divide used was a protected divide.

4 Initial population generation

To test which initial population generation was most beneficial to this dataset, each strategy was tested. The median MSE of the population for each run was taken and averaged across 5 runs for each strategy:

Averages:

- Full-grow: 0.02340896
- Grow: 0.03459818
- Ramped half-and-half: 0.01678272

It seems that the extra diversity from the ramped half-and-half initialisation reduced the median MSE. With full-grow initialisation, trees have somewhat uniform shapes, which limits the region of the search space that is explored. Grow seemed to lack diversity, which may be due to the fact that fewer trees are fully initialised, which limits the richness in the structure.

Strategy	Run	Median MSE	Seed
200 full-grow	1	0.0381856	5200
200 full-grow	2	0.0197934	5250
200 full-grow	3	0.0209398	7350
200 full-grow	4	0.0152003	8888
200 full-grow	5	0.0229257	9500
200 grow	1	0.0914435	5200
200 grow	2	0.0245564	7350
200 grow	3	0.0155205	8888
200 grow	4	0.0256638	9500
200 grow	5	0.0158067	5250
100 full-grow, 100 grow	1	0.0159178	5200
100 full-grow, 100 grow	2	0.0161455	7350
100 full-grow, 100 grow	3	0.0158067	8888
100 full-grow, 100 grow	4	0.0131205	9500
100 full-grow, 100 grow	5	0.0229231	5250

Table 1: Median MSE obtained for different initial population strategies across five runs.

5 Fitness function and fitness evaluation

Mean squared error was chosen over mean absolute error to penalise large errors more aggressively. A random sample of 5000 rows from the dataset was used to calculate the error (in the training, validation, and test sets) and evaluate fitness. This was due to the fact that there were 140,000 training cases, and 30 000 validation and test cases. The error was simply the sum of the differences squared between the individuals output and the target output in the training, validation and test sets.

6 Noise floor of the data and target MSE

To give my GP a target mean squared error, I estimated the noise floor of the dataset. This was done by calculating the k nearest neighbour of each row in the dataset (according to all the lag loads and previous day load). The mean of these variance values represents the noise floor.

Set	**Noise floor (MSE)**
Training	0.007328
Validation	0.015923
Test	0.082083
Entire set	0.009723

The *highestHitError* (target MSE) parameter was used to control what the system records as accurate given the noise floor. This was calculated to be 0.012

(adding 15% to the noise floor of the entire set). It follows that if an individual produces any result within 15% of the noise floor on the test set, it counts as a hit.

7 Selection method

Tournament selection was used, since it allows one to easily control the selection pressure. This problem benefited from a low selection pressure, which is more difficult to achieve with other selection methods. Convergence to a particular equation form was likely in this particular problem, and emphasis was placed on preventing premature convergence. A small *TournamentSize* (4), along with quite a large *PopulationSize* (640) was used.

8 Genetic operators

8.1 Crossover

To produce 2 new children in the next generation, two random points were used, and two random crossover nodes were chosen. Nodes were collected into an array before selection to ensure each crossover point has the same likelihood of getting chosen. The probability of crossover is determined by the *CrossoverRate* parameter.

8.2 Mutation

For mutation, a random node was selected. If a constant node was chosen as the random node, a coin was flipped according to the *TuneConstantProbability* parameter. If this succeeds, a small delta is added or subtracted from the constant node. This is because this problem lends itself to trying out many different constant values. If an operator or variable node was selected, grow mutation was used (a new sub-tree was grown from that point until the specified *MaxDepth* parameter using the grow method).

8.3 Reproduction (elitism)

For each generation, the fittest individual found during selection is cloned directly into a random position of the next generation to prevent that individual being lost due to crossover or mutation.

9 Experimental setup

9.1 Data set

The dataset was split up as follows:
Training set: 70%

Validation set: 15%
Test set: 15%.

The data was not shuffled, meaning that the validation set tested the models ability to predict unseen data "in the future", and the test set was further "in the future" (the most recent data). This allowed forecasting to occur.

The training set was used to evolve the population, while the validation set was used to test the quality of the final population. Finally, the best seeds and parameters were taken from the validation set and tested on the test set.

9.2 Parameters

9.3 Machine specifications

- **Model:** HP EliteBook 850 G8 Notebook PC
- **Processor:** Intel Core i7-1165G7
  - **Generation:** 11th Gen Intel Core
  - **Cores / Threads:** 4 cores / 8 threads
  - **Max Clock:** up to 4.70 GHz
- **GPU:**
  - **Graphics:** Intel Iris Xe Graphics (integrated)
  - **GPU Clock:** ~1.30 GHz
- **RAM**: 16 GB

10 Results

After the top 10 seeds from the validation set was chosen (according to the MSE obtained by the top individual on the validation set), these seeds were used to evolve the population and test it on the test set.

Below is the best, median and standard deviation mean squared error for the validation and test sets.

The runtime (to train the population) and the best individual node count is also included.

Seed	Val Best MSE	Val Med MSE	Val Avg MSE	Val StdDev MSE	Test Best MSE	Test Median MSE	Test StdDev MSE	Runtime (Sec)	Best Node Count
2822	**0.0095**	0.0210	3174320.0000	27570800.0000	**0.0109**	0.0442	0.0166	11.9720	33
2275	0.0097	0.0251	**1.1678**	8.2211	0.0112	0.0544	0.0216	9.7065	15
2939	0.0097	0.0170	5718.8800	141422.0000	0.0112	**0.0112**	**0.0000**	10.4982	16
4061	0.0097	0.0178	86.1830	2156.8400	0.0112	0.5881	0.2885	12.2940	17
2714	0.0098	0.0143	4.0737	43.6612	0.0112	0.1812	0.0850	11.1119	14
4080	0.0098	0.0162	18377.0000	463888.0000	0.0112	0.0199	0.0044	10.5387	13
1875	0.0098	0.0223	1840.3600	45719.3000	0.0112	0.0216	0.0052	14.6687	51
3293	0.0098	0.0213	6973.0200	161143.0000	0.0112	0.0112	**0.0000**	14.3377	17
1557	0.0098	0.0159	5.1527	79.9753	0.0113	0.0291	0.0089	6.9494	22
1711	0.0099	**0.0112**	0.5766	**3.5407**	0.0113	0.0113	**0.0000**	11.1234	17
Average	**0.0097**	**0.0182**	**320732.6414**	**2838526.4538**	**0.0112**	**0.0972**	**0.0430**	**11.3200**	**21.5000**

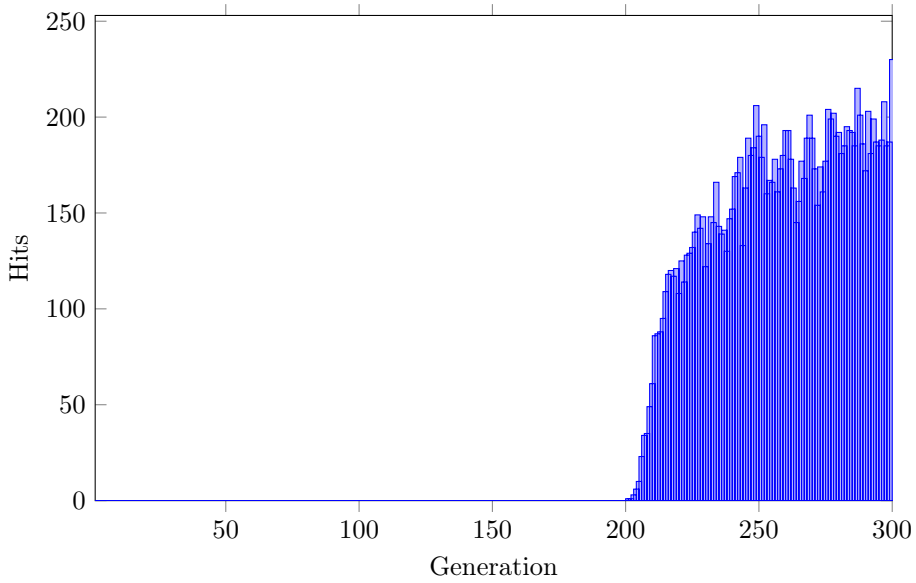
The individual with the lowest mean squared error for the test set was 0.0109. This is well within 15% of the noise floor, and seems to be close to the theoretical limit of the dataset. It also seems that the median mean squared error is low enough to signify a "healthy" population by the end of the run. Due to outliers, the average MSE was skewed in a couple of runs, which is to be expected. The average runtime is 21.5s, which is relatively fast.

For seed 2822, this is the individual produced:

```
DIV(
  ADD(
    ADD( ADD(x3,x0),
         ADD( ADD(x4(0.349315),x1), x6 ) ),
    ADD(
      ADD(
        DIV(
          ADD( ADD( ADD(x3,x0), ADD(x0,x6) ), x0 ),
          SQUARE(SUB(-0.499648,1.966100))
        ),
        x2
      ),
      x5
    )
  ),
  SQUARE(SUB(-0.762114,2.045397))
)
```

Many of the variables were used: x0 (load at t-1), x1(load at t-2), x3 (load at t-4), x6 (load at the same time the previous day). This suggests that the daily cycle and recent loads were the most effective inputs to make a prediction. Effectively, the model seems to be assigning different weights to the recent loads to make the prediction.

This is the hits per generation of the run that produced the individual with the best test MSE:



Before the population converges and starts to quickly gain progress, there are no hits. After some time, the GP discovers the "general form" of the equation and begins modifying constants to better fit the dataset.